Module 2 - ANSI SQL Using MySQL

NAME : MANOSHANKARI V

SUPERSET : 6736702

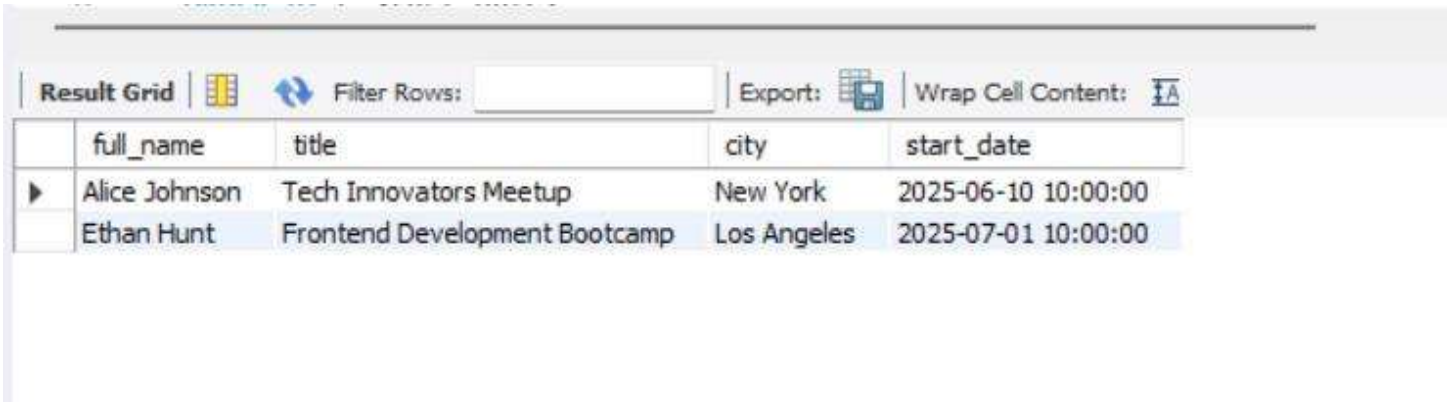
1. User Upcoming Events

Objective: Retrieve and filter user-specific event information using joins, conditions, and sorting.

Query:

```
SELECT u.full_name, e.title, e.city, e.start_date
FROM Users u
JOIN Registrations r ON u.user_id = r.user_id
JOIN Events e ON r.event_id = e.event_id
WHERE e.status = 'upcoming'
 AND u.city = e.city
ORDER BY e.start_date;
```

Output:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	title	city	start_date
Alice Johnson	Tech Innovators Meetup	New York	2025-06-10 10:00:00
Ethan Hunt	Frontend Development Bootcamp	Los Angeles	2025-07-01 10:00:00

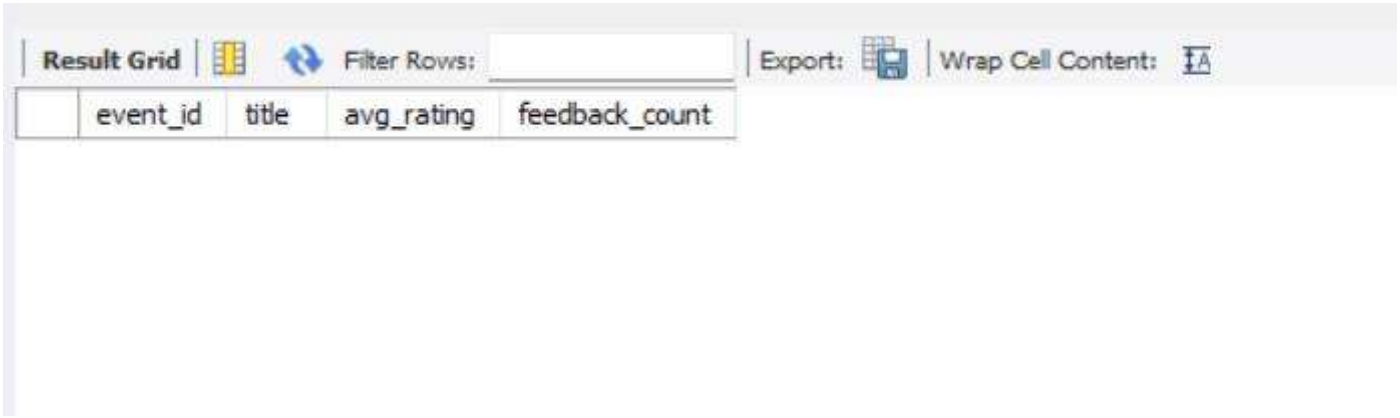
2. Top Rated Events

Objective: Calculate average ratings using aggregate functions and filter grouped results using HAVING.

Query:

```
SELECT e.title,
    AVG(f.rating) AS avg_rating,
    COUNT(*) AS feedback_count
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(*) >= 10
ORDER BY avg_rating DESC;
```

Output:



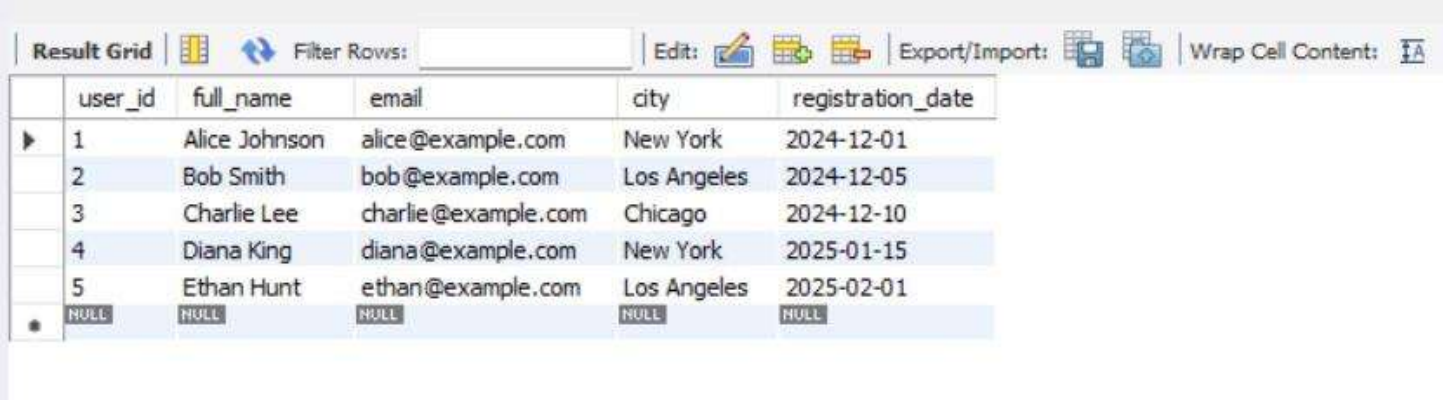
3. Inactive Users

Objective: Identify users with no recent activity using date functions and subqueries.

Query:

```
SELECT *
FROM Users u
WHERE u.user_id NOT IN (
  SELECT DISTINCT user_id
  FROM Registrations
  WHERE registration_date >= CURDATE() - INTERVAL 90 DAY
);
```

Output:



user_id	full_name	email	city	registration_date
1	Alice Johnson	alice@example.com	New York	2024-12-01
2	Bob Smith	bob@example.com	Los Angeles	2024-12-05
3	Charlie Lee	charlie@example.com	Chicago	2024-12-10
4	Diana King	diana@example.com	New York	2025-01-15
5	Ethan Hunt	ethan@example.com	Los Angeles	2025-02-01
NULL	NULL	NULL	NULL	NULL

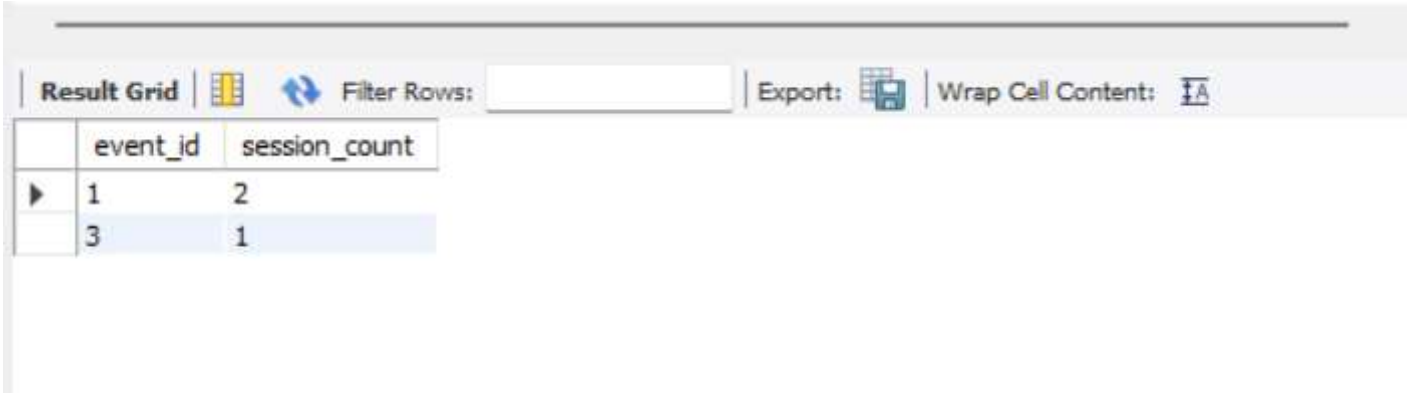
4. Peak Session Hours

Objective: Analyze session schedules using time-based filtering and grouping.

Query:

```
SELECT event_id,
    COUNT(*) AS session_count
FROM Sessions
WHERE TIME(start_time) BETWEEN '10:00:00' AND '12:00:00'
GROUP BY event_id;
```

Output:



event_id	session_count
1	2
3	1

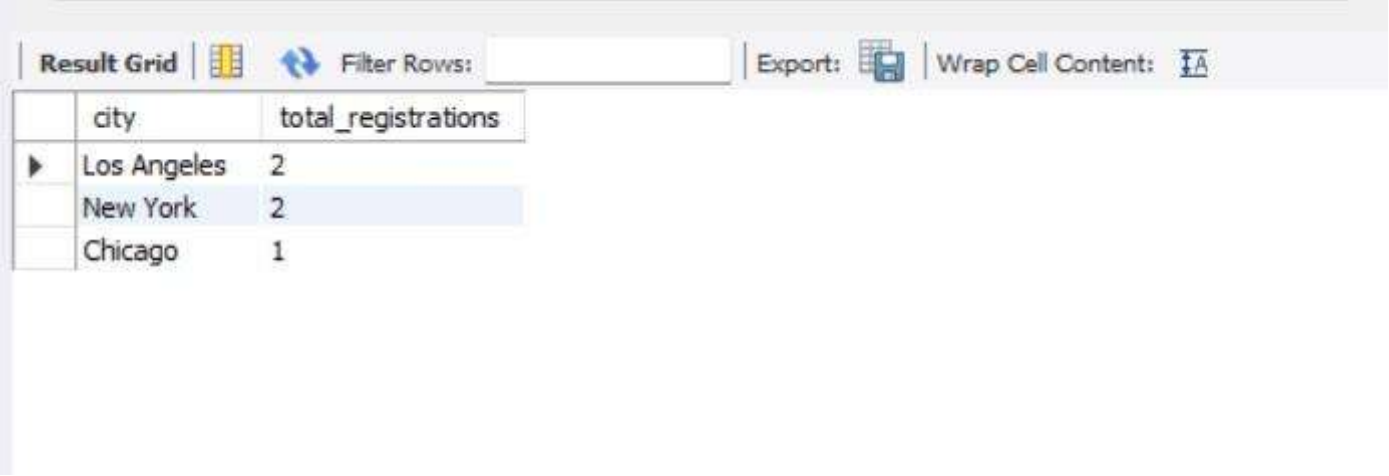
5. Most Active Cities

Objective: Determine registration activity by location using grouping and distinct counts.

Query:

```
SELECT e.city,
    COUNT(DISTINCT r.user_id) AS registrations
FROM Registrations r
JOIN Events e ON r.event_id = e.event_id
GROUP BY e.city
ORDER BY registrations DESC
LIMIT 5;
```

Output:



city	total_registrations
Los Angeles	2
New York	2
Chicago	1

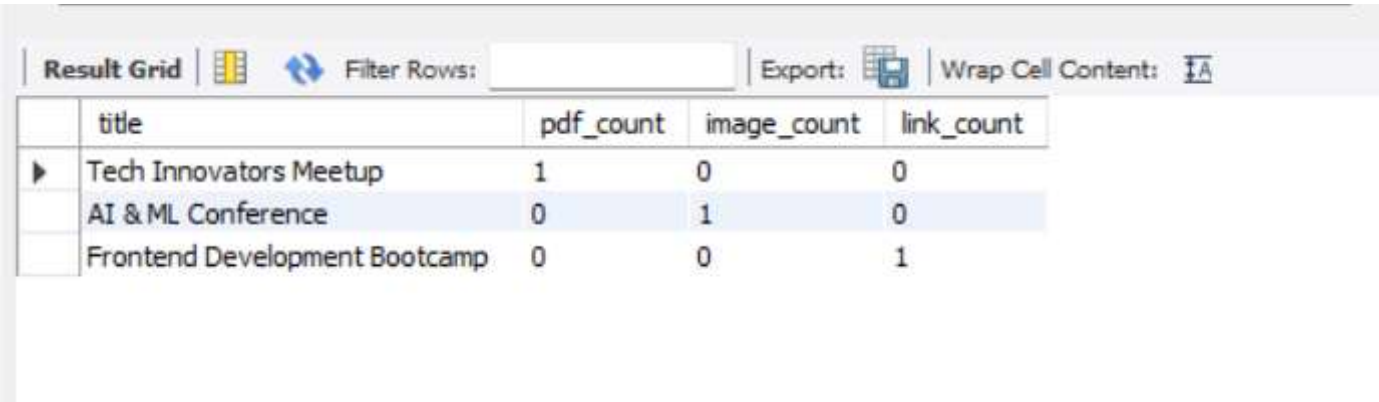
6. Event Resource Summary

Objective: Generate summarized reports using aggregate functions and grouping.

Query:

```
SELECT e.title,
    COUNT(r.resource_id) AS total_resources
FROM Events e
LEFT JOIN Resources r
ON e.event_id = r.event_id
GROUP BY e.event_id, e.title;
```

Output:



title	pdf_count	image_count	link_count
Tech Innovators Meetup	1	0	0
AI & ML Conference	0	1	0
Frontend Development Bootcamp	0	0	1

7. Low Feedback Alerts

Objective: Retrieve and analyze negative feedback using joins and conditional filtering.

Query:

```
SELECT u.full_name,
    e.title,
    f.rating,
    f.comments
FROM Feedback f
JOIN Users u ON f.user_id = u.user_id
JOIN Events e ON f.event_id = e.event_id
WHERE f.rating < 3;
```

Output:

8. Sessions per Upcoming Event

Objective: Count related records using joins and aggregate functions.

Query:

```
SELECT e.title,
    COUNT(s.session_id) AS total_sessions
FROM Events e
LEFT JOIN Sessions s
ON e.event_id = s.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title;
```

Output:

title	total_sessions
Tech Innovators Meetup	2
Frontend Development Bootcamp	1

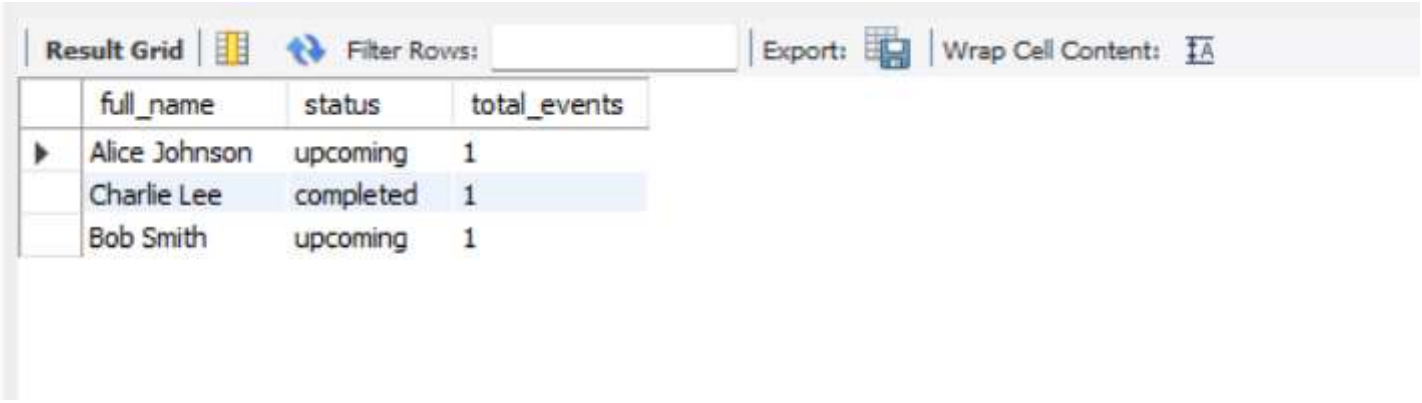
9. Organizer Event Summary

Objective: Summarize organizer performance by analyzing event creation and status distribution.

Query:

```
SELECT u.full_name,
    e.status,
    COUNT(*) AS event_count
FROM Events e
JOIN Users u
ON e.organizer_id = u.user_id
GROUP BY u.full_name, e.status;
```

Output:



full_name	status	total_events
Alice Johnson	upcoming	1
Charlie Lee	completed	1
Bob Smith	upcoming	1

10. Feedback Gap

Objective: Identify events with registrations but no feedback using outer joins and NULL checks.

Query:

```
SELECT DISTINCT e.title

FROM Events e

JOIN Registrations r ON e.event_id = r.event_id

LEFT JOIN Feedback f ON e.event_id = f.event_id

WHERE f.feedback_id IS NULL;
```

Output:

11. Daily New User Count

Objective: Analyze daily user registration trends using date grouping functions.

Query:

```
SELECT registration_date,
    COUNT(*) AS users_registered
FROM Users
WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY registration_date;
```

Output:

registration_date	users_registered

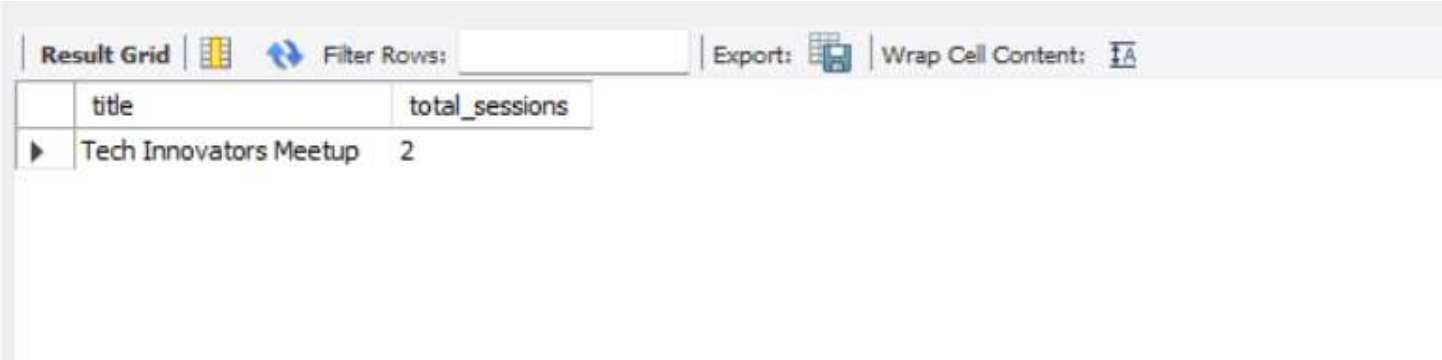
12. Event with Maximum Sessions

Objective: Find records with the highest aggregate values using subqueries or ranking techniques.

Query:

```
SELECT e.title, COUNT(s.session_id) AS total_sessions
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(s.session_id) = (
  SELECT MAX(cnt)
  FROM (
    SELECT COUNT(*) cnt
    FROM Sessions
    GROUP BY event_id
  ) x
);
```

Output:



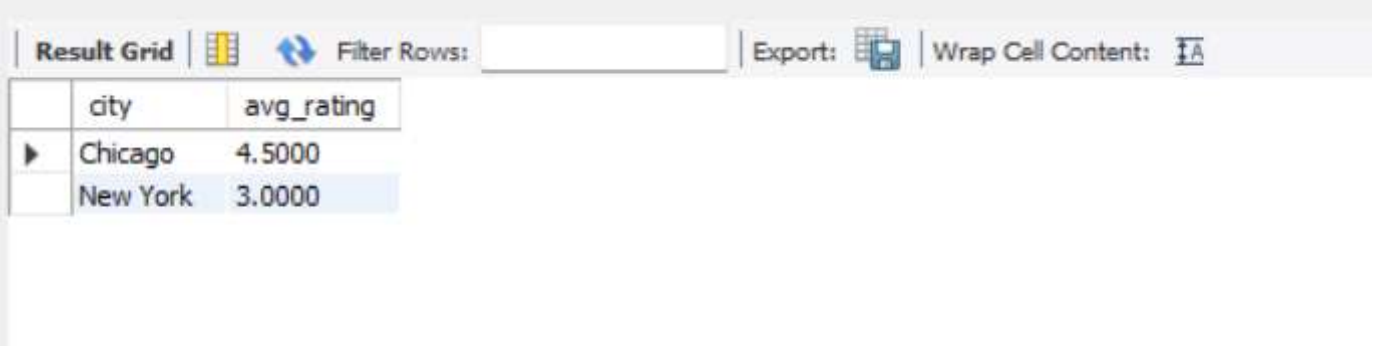
13. Average Rating per City

Objective: Calculate city-wise performance metrics using joins and aggregation.

Query:

```
SELECT e.city,
    AVG(f.rating) AS avg_rating
FROM Events e
JOIN Feedback f
ON e.event_id = f.event_id
GROUP BY e.city;
```

Output:



city	avg_rating
Chicago	4.5000
New York	3.0000

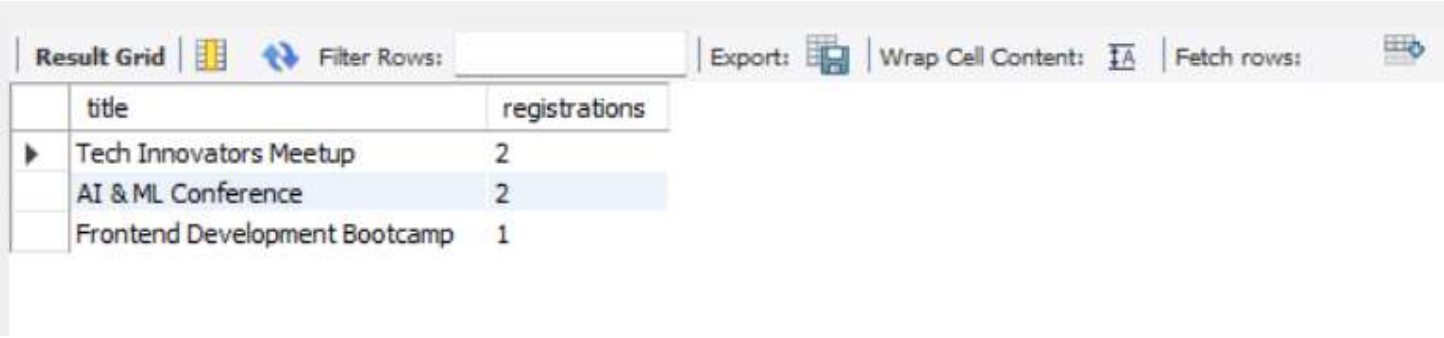
14. Most Registered Events

Objective: Rank events based on registration counts using sorting and limiting results.

Query:

```
SELECT e.title,
    COUNT(r.registration_id) AS registrations
FROM Events e
JOIN Registrations r
ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY registrations DESC
LIMIT 3;
```

Output:



title	registrations
Tech Innovators Meetup	2
AI & ML Conference	2
Frontend Development Bootcamp	1

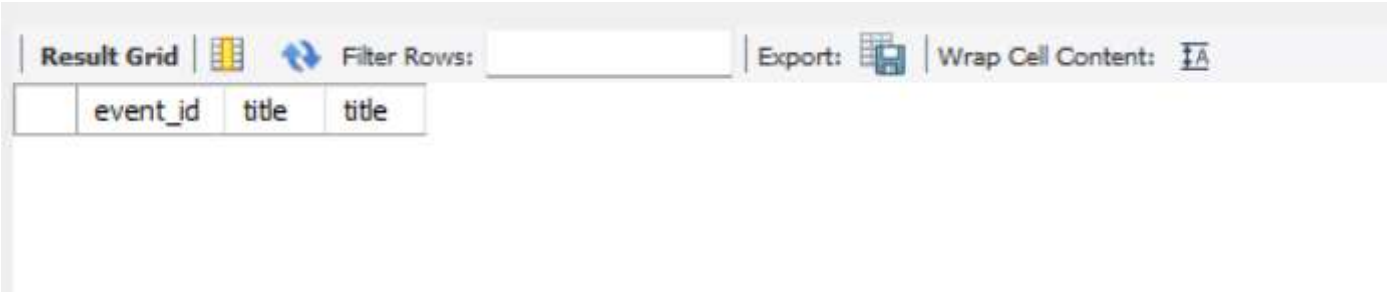
15. Event Session Time Conflict

Objective: Detect scheduling conflicts through date-time comparisons and self-joins.

Query:

```
SELECT s1.event_id,
    s1.title,
    s2.title
FROM Sessions s1
JOIN Sessions s2
ON s1.event_id = s2.event_id
AND s1.session_id < s2.session_id
AND s1.start_time < s2.end_time
AND s2.start_time < s1.end_time;
```

Output:



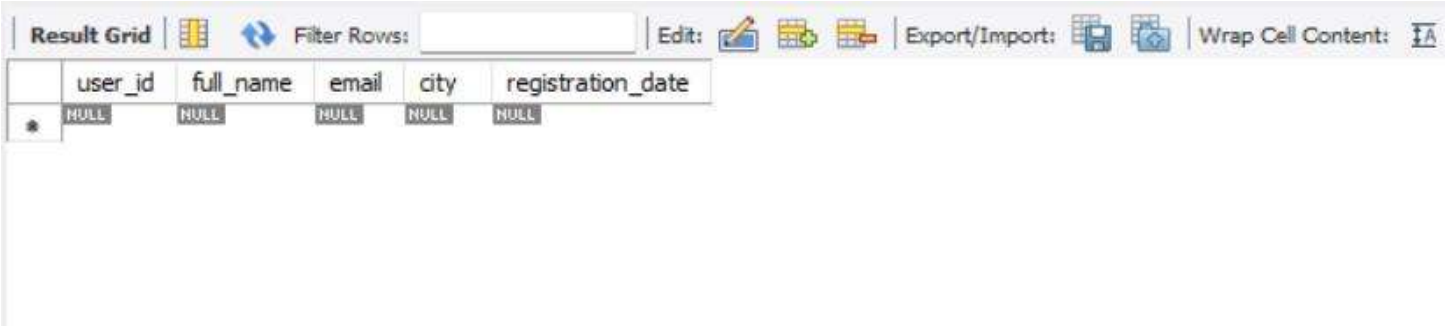
16. Unregistered Active Users

Objective: Identify recently registered users who have not participated in any events.

Query:

```
SELECT *
FROM Users u
WHERE registration_date >= CURDATE() - INTERVAL 30 DAY
AND NOT EXISTS (
  SELECT 1
  FROM Registrations r
  WHERE r.user_id = u.user_id
);
```

Output:



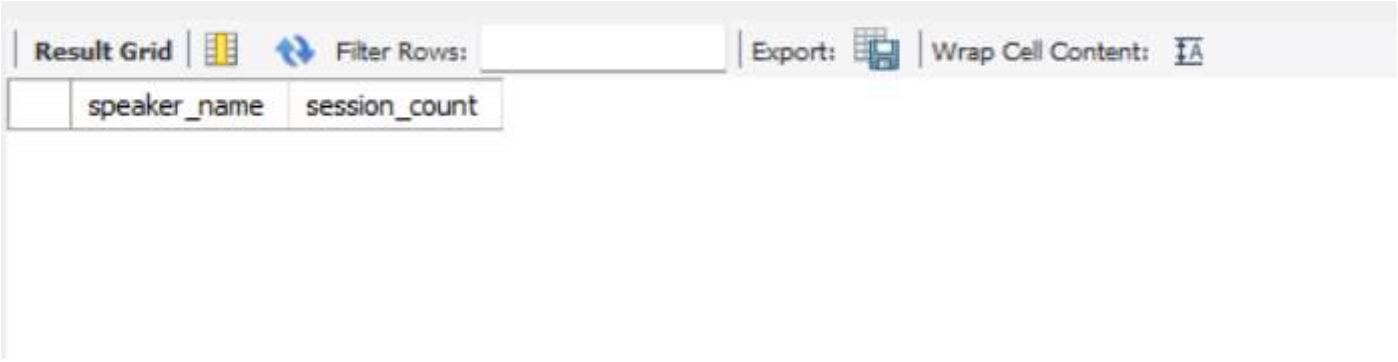
17. Multi-Session Speakers

Objective: Analyze speaker participation using grouping and count-based filtering.

Query:

```
SELECT speaker_name,
    COUNT(*) AS session_count
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(*) > 1;
```

Output:



18. Resource Availability Check

Objective: Find events lacking associated resources using outer joins and NULL conditions.

Query:

```
SELECT e.title

FROM Events e

LEFT JOIN Resources r

ON e.event_id = r.event_id

WHERE r.resource_id IS NULL;
```

Output:

19. Completed Events with Feedback Summary

Objective: Generate comprehensive reports combining registrations and feedback statistics.

Query:

```
SELECT e.title,
    COUNT(DISTINCT r.registration_id) AS registrations,
    AVG(f.rating) AS avg_rating
FROM Events e
LEFT JOIN Registrations r
ON e.event_id = r.event_id
LEFT JOIN Feedback f
ON e.event_id = f.event_id
WHERE e.status = 'completed'
GROUP BY e.event_id, e.title;
```

Output:

title	registrations	avg_rating
AI & ML Conference	2	4.5000

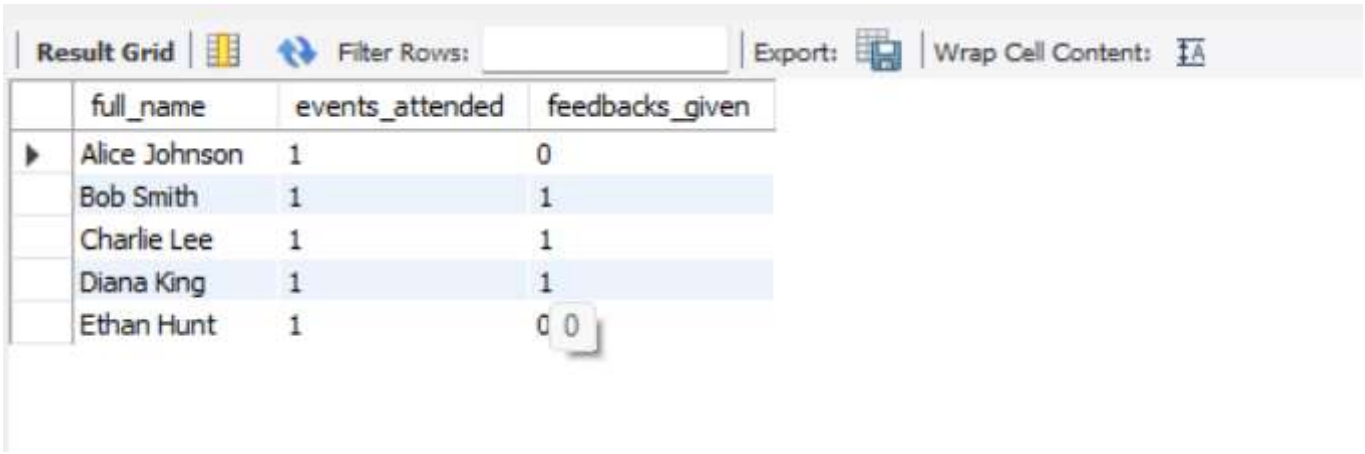
20. User Engagement Index

Objective: Measure user engagement by combining attendance and feedback metrics.

Query:

```
SELECT u.full_name,
    COUNT(DISTINCT r.event_id) AS events_attended,
    COUNT(DISTINCT f.feedback_id) AS feedbacks_given
FROM Users u
LEFT JOIN Registrations r
ON u.user_id = r.user_id
LEFT JOIN Feedback f
ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name;
```

Output:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	events_attended	feedbacks_given
Alice Johnson	1	0
Bob Smith	1	1
Charlie Lee	1	1
Diana King	1	1
Ethan Hunt	1	0

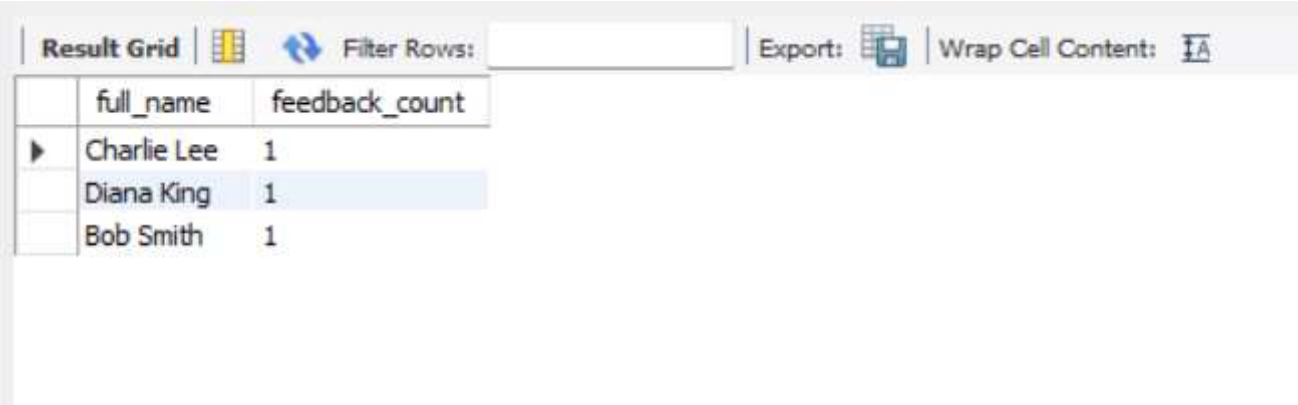
21. Top Feedback Providers

Objective: Identify highly active contributors using aggregation and ranking.

Query:

```
SELECT u.full_name,
    COUNT(*) AS feedback_count
FROM Users u
JOIN Feedback f
ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name
ORDER BY feedback_count DESC
LIMIT 5;
```

Output:



full_name	feedback_count
Charlie Lee	1
Diana King	1
Bob Smith	1

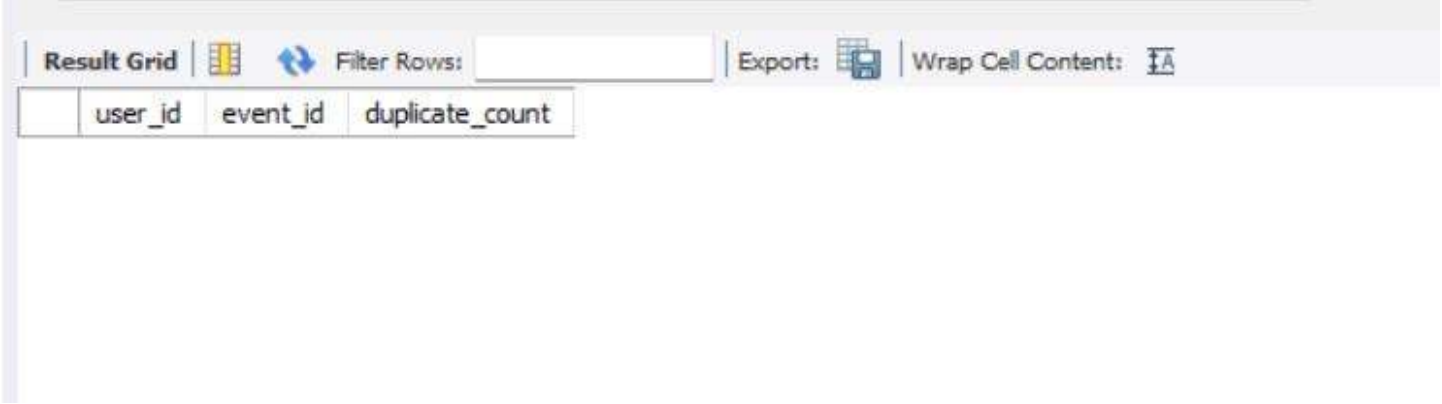
22. Duplicate Registrations Check

Objective: Detect data inconsistencies and duplicate records using grouping techniques.

Query:

```
SELECT user_id,
    event_id,
    COUNT(*) AS duplicate_count
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(*) > 1;
```

Output:



23. Registration Trends

Objective: Analyze month-wise registration patterns using date functions and aggregation.

Query:

```
SELECT DATE_FORMAT(registration_date,'%Y-%m') AS month,
    COUNT(*) AS registrations
FROM Registrations
WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH
GROUP BY month
ORDER BY month;
```

Output:

month	registrations
2025-06	1

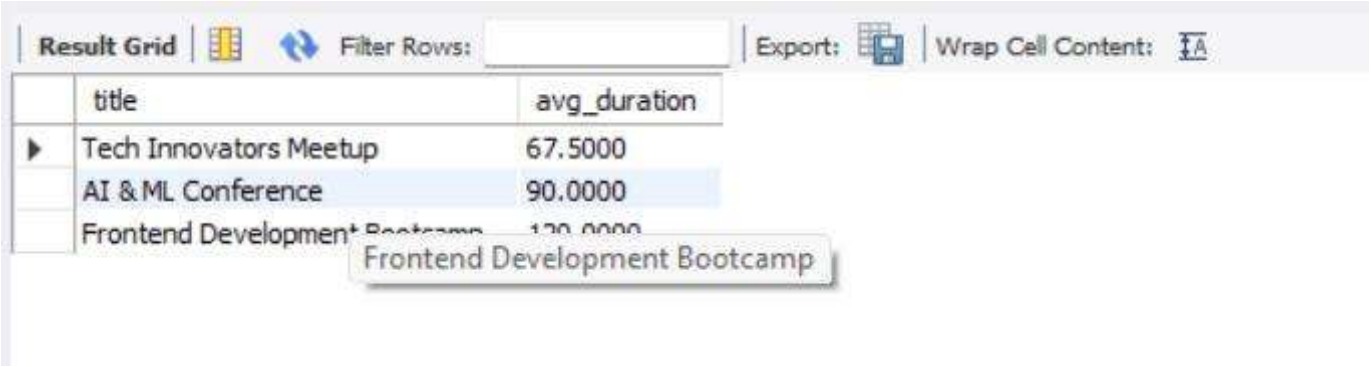
24. Average Session Duration per Event

Objective: Calculate average session durations using date/time functions and aggregate calculations.

Query:

```
SELECT e.title,
    AVG(TIMESTAMPDIFF(MINUTE,
            s.start_time,
            s.end_time)) AS avg_duration
FROM Events e
JOIN Sessions s
ON e.event_id = s.event_id
GROUP BY e.event_id, e.title;
```

Output:



25. Events Without Sessions

Objective: Identify events that have no associated session records using joins and NULL checks.

Query:

```
SELECT e.title

FROM Events e

LEFT JOIN Sessions s

ON e.event_id = s.event_id

WHERE s.session_id IS NULL;
```

Output: